

TRIBHUVAN UNIVERSITY

Faculty of Science and Technology
Mount Annapurna Campus, Pokhara

PROJECT REPORT · 2026

CUSTOM TAILORING MANAGEMENT SYSTEM

A Web-Based Platform for Custom Garment Selection,
Ordering and Tailoring Operations

A Project Report

Submitted in Partial Fulfillment of the Requirements for the
Bachelor of Science in Computer Science and Information Technology

SUBMITTED BY

Gyan Sagar Adhikari

Raj Thakuri

Ishan Bhujel

POKHARA, NEPAL · 2026

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to everyone who supported the development of the Atelier Custom Tailoring Management System. We are thankful to our faculty and project supervisor for their guidance, constructive feedback, and encouragement throughout the planning, design, implementation, and review of this project.

We also acknowledge the Department of Computer Science and Information Technology at Mount Annapurna Campus for providing the academic foundation in software engineering, web development, database systems, and information security that helped us undertake this work. We are grateful to our friends, classmates, and prospective users whose feedback helped us improve the clarity and usability of the customer and administration interfaces. Finally, we thank our families for their patience and continued support.

ABSTRACT

Custom garment ordering involves more than selecting a ready-made product. Customers need to coordinate a garment style, a compatible design, a fabric, measurements, fit preferences, and delivery details. When these details are handled informally, prices and order information can be difficult to track consistently. The Atelier Custom Tailoring Management System provides a web-based workflow for presenting tailoring options, collecting customer measurements and order instructions, managing production status, and recording payment information.

The system consists of a React single-page application and an ASP.NET Core REST API backed by PostgreSQL through Entity Framework Core. Customers can browse garment, design, and fabric catalogues, maintain a cart in the browser, submit an order, and view its progress. Administrators manage catalogue records and order status through a protected dashboard. The backend calculates each order using its stored catalogue prices and a delivery fee, and supports eSewa ePay v2 payment initiation and authoritative payment verification. The frontend submits gateway form fields returned by the backend and does not generate the eSewa signature.

This report presents the project objectives and scope, architecture, database model, functional modules, APIs, order and payment workflows, security controls, deployment configuration, validation performed, and limitations identified in the current implementation. It is based on the source code reviewed for this report; deployment state and external provider configuration may change independently.

TABLE OF CONTENTS

SECTION	PAGE
Acknowledgement	2
Abstract	3
List of Tables	5
Chapter 1: Introduction	6
Chapter 2: Objectives	7
Chapter 3: Scope of the Project	8
Chapter 4: Technology Stack	9
Chapter 5: System Architecture	10
Chapter 6: Database Design	11
Chapter 7: Modules and Features	12
Chapter 8: API Reference	13
Chapter 9: Order Lifecycle and Pricing	14
Chapter 10: eSewa Payment Integration	15
Chapter 11: Frontend Design	16
Chapter 12: Security Features	17
Chapter 13: Deployment and Configuration	18
Chapter 14: Testing and Validation	19
Chapter 15: Limitations and Recommended Improvements	20
Chapter 16: Conclusion	21
References	22

LIST OF TABLES

- Table 1: Technology stack
- Table 2: Main database entities
- Table 3: Customer and administration modules
- Table 4: Principal API endpoints
- Table 5: Order status transitions
- Table 6: Order price components
- Table 7: Important deployment configuration
- Table 8: Validation scenarios

CHAPTER 1

INTRODUCTION

Clothing is personal, and customers looking for tailored garments often need to communicate style, fabric, fit, and measurement requirements before production can begin. A fragmented process based on calls or messages can make it difficult for customers to compare options and for a tailoring business to maintain a consistent record of the requested work.

Atelier is a web application designed to organize these steps. Its customer interface presents garment, design, and fabric options and provides a path from product exploration to checkout and order tracking. Its administration area gives staff a place to maintain the catalogue and move customer orders through defined production statuses. A backend API enforces ownership, pricing, and payment rules and persists records in a relational database.

The product is referred to as Atelier or Atelier Rouge in parts of the client interface. This report uses Atelier as the system name. It reflects the current codebase and separates implemented behavior from future recommendations.

CHAPTER 2

OBJECTIVES

The project aims to deliver the following outcomes:

- Provide a responsive storefront for browsing garments, designs, and fabrics.
- Allow a customer to assemble a custom garment request with a design, fabric, measurements, fit preference, shipping details, and optional instructions.
- Persist orders and customer-specific measurement profiles through an authenticated API.
- Give administrators protected tools to maintain garments, designs, fabrics, and order workflow status.
- Calculate order amounts on the backend from persisted catalogue prices and defined fees.
- Integrate eSewa payment initiation and verification without exposing gateway signing logic in the browser.
- Present customers with their order history, order details, and payment confirmation state.
- Document the architecture and provide a foundation for maintainable deployment and future feature work.

CHAPTER 3

SCOPE OF THE PROJECT

Included in the current implementation

- Customer registration and login with JWT-based access to protected operations.
- Public browsing of active garment, design, and fabric records.
- Product detail, cart, and checkout experiences; the current cart is stored in browser local storage.
- Measurement-profile creation associated with the authenticated customer and selected garment.
- Order placement, customer order listing, order detail retrieval, and cancellation while the order remains Pending.
- Admin dashboard areas for garment, design, fabric, and order management.
- Order status history and backend validation of allowed status transitions.
- eSewa payment form handoff, callback processing, signature verification, and server-side status confirmation.
- Image upload through the configured Cloudinary unsigned upload preset.

Outside the current scope

The current frontend checkout flow initiates eSewa payment. Although the backend includes Cash on Delivery handling, the current customer checkout interface does not provide a complete payment-method selection experience. The backend project also contains review-related entities and DTOs, but no active review controller is present in the inspected route set. Persistent server-side cart management, appointment scheduling, inventory reservation, delivery-provider integration, and production analytics are not represented as complete end-to-end features.

CHAPTER 4

TECHNOLOGY STACK

Layer	Technology	Purpose
Customer/admin frontend	React 19, JavaScript and selected TypeScript components	Single-page interface and reusable page components
Build and styling	Vite 8, Tailwind CSS 4	Development server, production build and utility styling
Client data and routing	Redux Toolkit, RTK Query, React Router 7	Shared state, API requests, caching and client-side routes
Backend API	ASP.NET Core 10 / C#	REST endpoints, authorization and business rules
Persistence	Entity Framework Core 10, Npgsql	Object-relational access to PostgreSQL
Database	PostgreSQL (Neon in the current configuration)	Identity, catalogue, measurement, order and payment records
Payments	eSewa ePay v2	Hosted payment form and payment status verification
Product imagery	Cloudinary unsigned image upload	Browser image upload and hosted image URLs
Hosting configuration	Vercel frontend; Docker-based Render backend planned	Static SPA hosting and containerized API hosting

CHAPTER 5

SYSTEM ARCHITECTURE

The application follows a client–API–database architecture. The React client calls the ASP.NET Core API over HTTPS. The API authenticates customers with bearer JWTs, checks authorization for protected routes, applies business rules in service classes, and reads or writes PostgreSQL through Entity Framework Core. Product image bytes are uploaded from the browser to Cloudinary using the configured unsigned upload preset; the resulting URL is saved with catalogue data through the API.

Component	Responsibilities
React SPA	Renders customer storefront and admin dashboard; manages routing, local cart, form state and API interaction.
ASP.NET Core API	Authenticates users; exposes catalogue, measurement, order and payment endpoints; enforces authorization and business rules.
PostgreSQL database	Stores Identity users/roles, catalogue records, measurement profiles, orders, status history and payments.
eSewa gateway	Hosts payment page and returns a callback payload to the backend.
Cloudinary	Receives image uploads and serves image URLs used in the catalogue.

A typical checkout begins in the frontend, which creates a measurement profile and then an order. The backend computes and persists the total. The frontend then asks the backend to initiate payment. For eSewa, the backend returns the gateway URL and signed form fields; the browser posts those fields as an HTML form to the hosted gateway. The backend handles the gateway callback and verifies payment data before recording success.

CHAPTER 6

DATABASE DESIGN

The database uses PostgreSQL and ASP.NET Core Identity. Entity relationships and column types are defined in ApplicationDbContext. JSONB is used for structured measurement snapshots, customization details, garment measurement requirements, and raw gateway response data. Monetary and quantity columns use explicit decimal precision. Enumerated order and payment values are persisted as strings.

Entity / table concept	Purpose and relationships
ApplicationUser / Identity	Customer/admin accounts and role membership; users own measurements, orders and payments.
Garment	Base garment definitions and required measurement metadata; related to designs and measurements.
Design	Tailoring/design option associated with a garment; stores tailoring price and image.
Fabric	Fabric catalogue, unit price, image, active state and available quantity.
MeasurementProfile	Customer-owned measurement values linked to a garment; orders retain a measurement snapshot.
Order	Customer order, selected garment/design/fabric, pricing components, shipping details, status and payment status.
OrderStatusHistory	Timestamped order status changes and notes.
Payment	Payment transaction associated one-to-one with an order and linked to the customer.
ShopProfile / Review	Supporting domain entities exist in the model; review operations are not exposed by a current controller.

The model defines unique order numbers and a unique payment per order. Order-to-catalogue relationships restrict deletion, measurement profiles are user-owned, and order/payment relationships are configured explicitly. The project has an Entity Framework migration. The API does not automatically execute migrations during application startup.

CHAPTER 7

MODULES AND FEATURES

7.1 Authentication

Customers register and sign in through the authentication API. Successful authentication returns a JWT used in the Authorization bearer header for protected operations. Registration assigns the Customer role. The admin account and the Admin/Customer roles are initialized by AdminSetupService using configuration values.

7.2 Customer storefront and catalogue

The home and product pages surface tailoring garments and their related designs and fabrics. Product and catalogue queries are cached through RTK Query. Garment, design, and fabric records include names, prices, descriptions, active state, and imagery as supported by their models and DTOs.

7.3 Cart, measurements and checkout

The cart is persisted per user in browser local storage. The current checkout supports one distinct garment per order, creates a measurement profile from the selected garment's measurement snapshot, and submits selected catalogue IDs and shipping/order details to the backend. The customer's cart is cleared after successful order creation and payment initiation.

7.4 Customer orders

Authenticated customers can list their own orders and view an individual order with garment, design, fabric, measurement, customization, pricing, payment, and status information. Customers can cancel an order only while its status is Pending.

7.5 Administration

Admin-only pages manage garment, design, and fabric catalogue records, including image previews/uploads. The order-management screen supports filtering and viewing orders. It offers only the next status transitions accepted by the backend and requires a rejection reason when rejecting an order; changing a selection alone does not update the record until confirmed.

7.6 Payments

The backend supports eSewa initiation, verification, status checks, and Cash on Delivery records. The customer-facing checkout currently directs the online checkout path to eSewa. Payment status is maintained separately from production order status.

CHAPTER 8

API REFERENCE

The principal routes below are derived from the backend controllers. Unless stated as public, routes under Order, MeasurementProfile and Payment require a valid bearer JWT. Catalogue reads are public; catalogue write operations and all admin order operations require the Admin role.

Area	Method and route	Access / purpose
Authentication	POST /api/auth/register	Public; create a customer account.
Authentication	POST /api/auth/login	Public; issue authentication token.
Authentication	GET /api/auth/profile	Authenticated; retrieve own profile.
Garments	GET /api/garment/{id}	Public catalogue list or item detail.
Garments	POST, PUT /api/garment/{id}; DELETE /api/garment/{id}	Admin catalogue maintenance.
Designs	GET /api/design/{id}	Public catalogue list or item detail.
Designs	POST, PUT /api/design/{id}; DELETE /api/design/{id}	Admin catalogue maintenance.
Fabrics	GET /api/fabric/{id}	Public catalogue list or item detail.
Fabrics	POST, PUT /api/fabric/{id}; DELETE /api/fabric/{id}	Admin catalogue maintenance.
Measurements	GET/POST /api/measurementprofile; GET/PUT/DELETE /api/measurementprofile/{id}	Authenticated measurement profile operations.
Customer orders	GET /api/order; GET /api/order/{id}	Authenticated customer's own order list and detail.
Customer orders	POST /api/order; PATCH /api/order/{id}/cancel	Authenticated order creation and Pending-only cancellation.
Admin orders	GET /api/admin/order; GET /api/admin/order/{id}	Admin order list and detail.
Admin orders	PATCH /api/admin/order/{id}/status	Admin status transition; invalid transitions return conflict.
Payment	POST /api/payment/initiate; POST /api/payment/verify	Authenticated initiation and verification.
Payment	POST /api/payment/eseva/status/{orderId}	Authenticated eSewa status check.
Payment callback	GET/POST /api/payment/eseva/success and /failure	Gateway callbacks; success callback is anonymous and verified server-side.

The server returns HTTP status codes including 401 for unauthenticated access, 404 for missing resources, and 409 for invalid business-state conflicts. Full DTO schemas should be consulted in Models/DTOs before building another API client.

CHAPTER 9

ORDER LIFECYCLE AND PRICING

9.1 Order creation

Order creation validates the fit and payment enums, positive fabric quantity, shipping address, JSON structure, customer ownership and active state of the selected garment, design and fabric. The design and measurement profile must belong to the selected garment. The service snapshots measurement values onto the order so later profile edits do not rewrite the order's submitted measurements.

9.2 Price calculation

Component	Backend calculation
Fabric	Fabric unit price $\times$ requested fabric quantity
Tailoring	Garment base price (or zero if absent) + design tailoring price
Customization	Currently fixed at Rs 0
Delivery	Currently fixed at Rs 100
Order total	Fabric + tailoring + customization + delivery

The backend-calculated and persisted total is authoritative for payment. The frontend shows an estimate based on catalogue data held in the cart; a changed catalogue price can make that estimate differ from the amount stored by the server.

9.3 Status transitions

Current status	Allowed next status
Pending	Accepted or Rejected
Accepted	InStitching
InStitching	Completed
Completed	Ready
Ready	Delivered
Rejected / Cancelled / Delivered	No further transition in the current transition map

The backend requires a non-empty rejection reason when moving an order to Rejected. Customer cancellation is permitted only from Pending. Payment statuses include Unpaid, PendingVerification, Paid, Failed, and Refunded and are independent of garment-production statuses.

CHAPTER 10

ESEWA PAYMENT INTEGRATION

The backend creates or updates the payment record using the persisted order amount and creates a unique transaction reference. It generates the eSewa signature using the server-side configured secret and returns the payment URL and hidden form fields to the browser. The frontend posts those fields to eSewa; it must not calculate or modify the signature and must never receive the secret key.

The eSewa callback is processed by the backend. Callback data is decoded, the signature is checked, the transaction reference/product code/status and amount are matched against the stored payment and order, and authoritative status-check operations are available. The customer success page fetches the order and only treats it as paid when the backend reports Paid; if required, it invokes backend verification and refreshes the order.

Customer submits checkout; backend persists an order with the authoritative total.

Frontend calls POST /api/payment/initiate for that existing order.

Backend returns returnUrl, payment details, transactionReference and signed formFields.

Frontend creates a hidden HTML form and posts it to the eSewa hosted form.

eSewa returns to the backend success/failure callback URLs.

Backend validates the callback and gateway status, then redirects the browser to the configured frontend result route.

Frontend displays success only after reading the backend order's Paid status.

Sandbox/test and live merchant credentials, endpoints, callback hostnames, and product codes must be kept consistent. Live payment must not be enabled until the merchant's production credentials and public HTTPS callbacks are configured and tested.

CHAPTER 11

FRONTEND DESIGN

The frontend is a React single-page application using React Router and shared layout components. Customer-facing pages include the home page, login, signup, product list and detail, cart, checkout, order history, order details, and payment success/failure pages. Admin pages are guarded by the role-aware `ProtectedRoute` and share an administrative shell and sidebar.

The visual language uses Atelier branding, responsive layouts, image-led product presentation, loading skeletons for data-backed views, and explicit error messages for API failures. The app's Vercel deployment requires a rewrite of unknown paths to `index.html` so that direct navigation and browser refreshes work with client-side routes. A Vercel rewrite configuration has been added to the frontend repository.

The frontend's `VITE_API_URL` selects the API base address at build time. Cloudinary cloud name and unsigned upload preset are also frontend-visible configuration values; they must not contain a Cloudinary API secret.

CHAPTER 12

SECURITY FEATURES

- JWT bearer authentication validates issuer, audience, token lifetime and signing key.
- Admin catalogue writes and administrative order routes require the Admin role.
- Customer order and measurement queries are scoped to the authenticated customer identity.
- Order creation validates related IDs, active catalogue records, quantities and measurement ownership server-side.
- The payment amount and eSewa signature are generated on the backend; payment callbacks are verified before marking a payment successful.
- CORS uses an explicit list of allowed origins and supports credentials; production origins must be configured precisely.
- Secret configuration is expected from the host environment/secret manager, not from frontend `VITE_*` variables.

Security work still recommended includes implementing a real refresh-token lifecycle or removing the client refresh attempt, strengthening the configured account password policy, introducing rate limits for public authentication endpoints, validating image upload constraints in Cloudinary, adding structured security logging, and restricting production AllowedHosts.

The frontend persists authentication tokens in browser local storage in the current implementation. This simplifies client session restoration but increases the impact of any cross-site scripting vulnerability. A hardened production design should assess short-lived access tokens and secure, HttpOnly cookie-based refresh tokens, alongside a strong content security policy and dependency review.

CHAPTER 13

DEPLOYMENT AND CONFIGURATION

The frontend is a Vite static build suitable for Vercel. The backend project targets .NET 10 and has a multi-stage Dockerfile prepared for a Render web service. The container binds to the PORT environment variable. The frontend URL supplied by the project team is <https://tailor-frontend-vert.vercel.app>; actual deployment health and API connectivity should be confirmed in the respective provider dashboards.

Backend environment settings

Environment key	Purpose / required value
ConnectionStrings__DefaultConnection	PostgreSQL connection string for the selected Neon database.
Jwt__Secret	Private random production signing secret.
Jwt__Issuer / Jwt__Audience	Must match API token issuance and validation configuration.
Admin__Email / Admin__FullName / Admin__Password	Bootstrap account identity and required startup password configuration.
Cors__AllowedOrigins__0	Exact browser origin, e.g. https://tailor-frontend-vert.vercel.app .
Frontend__BaseUrl	Frontend origin used for payment callback redirects.
Esewa__BackendBaseUrl	Public backend HTTPS origin used to construct eSewa callback URLs.
Esewa__PaymentUrl / StatusCheckUrl / ProductCode / SecretKey	Gateway endpoints, merchant code and private test/live secret.
ASPNETCORE_ENVIRONMENT	Production for the hosted deployment.

Secret values must be entered directly into Render’s environment settings and never committed to Git or included in the frontend bundle. Update Cors__AllowedOrigins__0 and Frontend__BaseUrl after the final Vercel domain is known. Replace sandbox eSewa credentials only after production merchant configuration is approved.

Database migrations and shared data

The backend checks its database connection during startup but does not automatically apply migrations. Back up the existing Neon database before any migration. Review the migration against the current schema and apply it as an intentional release operation; do not assume deploying the container migrates the database.

CHAPTER 14

TESTING AND VALIDATION

Validation performed during development included a successful frontend production build, ESLint on the pricing/cart/checkout changes, a backend .NET build and Release publish, a direct pricing-formula check, and Git whitespace checks. The frontend build emitted a bundle-size advisory; it completed successfully.

Scenario	Expected validation
Browse catalogue	Public API returns active garments, designs and fabrics; images load.
Protected customer request	Valid JWT succeeds; unauthenticated access is rejected.
Create order	Backend validates ownership and selection; persisted total matches stored catalogue prices and fees.
Invalid order transition	API returns conflict and leaves order unchanged.
eSewa initiation	Form fields use backend amount/reference/signature; browser redirects by form POST.
Payment return	Only server-verified Paid order is presented as successful.
Direct route reload	Vercel serves SPA fallback for a nested frontend route.
Production browser/API call	CORS permits the exact deployed frontend origin and rejects unrelated origins.

No dedicated automated unit- or end-to-end-test project was found in the inspected repositories. The scenarios above are recommended release checks, not a claim that an automated suite has executed them.

CHAPTER 15

LIMITATIONS AND RECOMMENDED IMPROVEMENTS

- Add automated backend unit/integration tests and browser-driven checkout/payment tests.
- Add a health endpoint and production observability for Render health checks and operational troubleshooting.
- Implement and test secure refresh-token issuance, rotation, revocation and storage; the frontend currently attempts a refresh route that is absent from the backend.
- Add password reset/change and email verification flows and strengthen password policy before public launch.
- Review database migration status and add a repeatable, controlled migration deployment step.
- Add a server-side cart if cross-device persistence or multi-item checkout is required; the current checkout supports one distinct garment per order.
- Add inventory reservation/adjustment rules and clarify that cart quantity maps to fabric quantity in the current order payload.
- Provide a supported customer payment-method selector if Cash on Delivery is intended to be a customer-facing option.
- Implement and expose review operations if reviews are part of the intended product.
- Constrain image upload preset formats, file size, folder and moderation policy; consider signed uploads if required.
- Optimize oversized images and split the frontend bundle to improve initial load performance.
- Add clear privacy, terms, refund, delivery and tailoring measurement guidance for real customers.

CHAPTER 16

CONCLUSION

The Atelier Custom Tailoring Management System brings the key parts of a custom-garment order into one web workflow: catalogue discovery, choice of tailoring options, measurement capture, order creation, production-status tracking, and online payment verification. The separate React client and ASP.NET Core API support a clear division between presentation and server-enforced business rules.

The implementation provides a functional MVP foundation, particularly in the admin-managed catalogue and order workflows. The backend remains authoritative for prices and payment status, which is essential for correct totals and trustworthy eSewa processing. Before the system is used for live customer payments, the team should complete deployment configuration, test the full production callback path, review the existing database and migrations, and address the security and test coverage recommendations in this report.

REFERENCES

1. Atelier project source code: React frontend repository and Atelier-backend ASP.NET Core repository, reviewed October 2026.
2. React documentation. <https://react.dev/>
3. Vite documentation. <https://vite.dev/guide/>
4. Redux Toolkit documentation. <https://redux-toolkit.js.org/>
5. React Router documentation. <https://reactrouter.com/>
6. Microsoft ASP.NET Core documentation. <https://learn.microsoft.com/aspnet/core/>
7. Entity Framework Core documentation. <https://learn.microsoft.com/ef/core/>
8. PostgreSQL documentation. <https://www.postgresql.org/docs/>
9. eSewa ePay developer documentation. <https://developer.esewa.com.np/>
10. Cloudinary upload documentation.
https://cloudinary.com/documentation/image_upload_api_reference
11. Vercel rewrites documentation. <https://vercel.com/docs/rewrites>
12. Render Docker documentation. <https://render.com/docs/docker>